

O capítulo “Design by Contract”, de Bertrand Meyer, apresenta uma proposta voltada para a construção de softwares mais confiáveis, corretos e robustos. Desde o início, o autor demonstra preocupação com um problema recorrente da engenharia de software: a dificuldade de produzir sistemas que funcionem de maneira segura e previsível. Para isso, ele introduz a ideia de “contrato” aplicada ao desenvolvimento de programas, utilizando uma metáfora inspirada nos contratos do mundo real para explicar como módulos de software devem interagir entre si. O texto também relaciona essa abordagem à programação orientada a objetos, especialmente dentro da linguagem Eiffel, destacando como contratos podem melhorar a reutilização de componentes e reduzir falhas em sistemas complexos.

Ao longo do capítulo, Meyer explica que o desenvolvimento de software pode ser entendido como uma sequência de decisões de terceirização de tarefas. Sempre que um programador cria uma rotina, ele precisa decidir se realizará determinada subtarefa diretamente ou se irá delegá-la a outro módulo. Essa ideia de “contratar” partes do sistema ajuda a organizar melhor o código e favorece a reutilização de soluções já existentes. Entretanto, o autor deixa claro que essa divisão só funciona corretamente quando as responsabilidades de cada parte estão bem definidas.

É justamente nesse ponto que surge o conceito central do texto: o contrato de software. Assim como em um contrato jurídico, ambas as partes possuem obrigações e benefícios. O cliente, que chama uma rotina, deve garantir certas condições antes da execução. Em troca, o fornecedor da rotina garante um resultado específico. Meyer utiliza o exemplo de uma construção civil para ilustrar essa lógica: um construtor só é obrigado a entregar o prédio se o cliente fornecer o terreno adequado. Da mesma forma, uma rotina só precisa funcionar corretamente quando suas condições de uso forem respeitadas.

Para tornar esses contratos explícitos dentro do software, o autor apresenta o uso das chamadas assertions, ou afirmações. Essas afirmações são divididas em pré-condições, pós-condições e invariantes de classe. As pré-condições representam aquilo que deve ser verdadeiro antes da execução de uma rotina. Já as pós-condições descrevem o que a rotina garante após sua execução. Os invariantes, por sua vez, representam regras gerais que devem permanecer verdadeiras durante toda a vida de um objeto.

O exemplo utilizado com a rotina de inserção em uma tabela ajuda bastante na compreensão desse mecanismo. A pré-condição exige que a tabela não esteja cheia antes da inserção de um novo elemento. Após a execução, a pós-condição garante que o item foi realmente armazenado e que a quantidade de elementos aumentou corretamente. O interessante é que Meyer não trata essas verificações apenas como ferramentas técnicas, mas também como instrumentos de documentação e comunicação entre programadores.

Um dos pontos mais relevantes do texto é a crítica à chamada programação defensiva. Tradicionalmente, muitos sistemas realizam verificações redundantes em praticamente todos os pontos do código, tentando prever qualquer erro possível. Para o autor, isso gera programas excessivamente complexos, difíceis de manter e menos confiáveis. Em vez disso, ele defende que cada responsabilidade seja claramente atribuída: ou o cliente garante determinada condição, ou o fornecedor assume o tratamento daquele caso. Nunca ambos ao mesmo tempo.

Essa discussão leva a uma reflexão importante sobre simplicidade arquitetural. Meyer argumenta que um software cheio de validações repetidas perde clareza e acaba dificultando a própria manutenção do sistema. Segundo ele, contratos bem definidos reduzem a necessidade de verificações desnecessárias e tornam o código mais limpo e compreensível. Essa visão demonstra uma preocupação não apenas com funcionamento técnico, mas também com qualidade estrutural do software.

Outro conceito fundamental apresentado é o de invariante de classe. O autor explica que certas propriedades precisam permanecer verdadeiras durante toda a existência de um objeto. No caso de uma tabela, por exemplo, o número de elementos nunca pode ser negativo nem ultrapassar sua capacidade máxima. Essas regras representam uma espécie de “estado válido” do objeto. Assim, qualquer rotina pública da classe deve preservar esse estado após sua execução.

O texto também aprofunda a relação entre contratos e corretude de software. Meyer afirma que um programa não pode ser considerado correto de forma absoluta, mas apenas em relação à sua especificação. Dessa forma, pré-condições, pós-condições e invariantes funcionam como uma definição formal do comportamento esperado do sistema. Essa ideia aproxima a engenharia de software de disciplinas mais rigorosas, nas quais é necessário provar matematicamente que determinado comportamento será respeitado.

Outro aspecto muito interessante do capítulo é a discussão sobre monitoramento de assertions em tempo de execução. O autor explica que violações de assertions indicam falhas de projeto, implementação ou especificação. Se uma pré-condição é violada, o erro pertence ao cliente da rotina; se uma pós-condição falha, o problema está na implementação do fornecedor. Isso cria uma separação extremamente clara de responsabilidades, algo que facilita tanto a depuração quanto a manutenção do sistema.

Além disso, Meyer apresenta uma reflexão bastante crítica sobre o tratamento tradicional de exceções em linguagens como Ada e CLU. Em diversos exemplos, ele mostra que exceções são frequentemente usadas de maneira inadequada, mascarando erros ou tornando o fluxo do programa mais confuso. O caso da função de raiz quadrada é especialmente marcante: quando um valor negativo é recebido, a rotina simplesmente exibe uma mensagem e continua a execução, como se nada tivesse acontecido. Para o autor, isso é perigoso, pois o sistema continua operando sem garantir que o contrato foi cumprido.

A partir dessa crítica, o texto propõe uma abordagem disciplinada para o tratamento de exceções. Meyer estabelece leis fundamentais para esse mecanismo. A primeira delas afirma que uma rotina só pode terminar de duas formas: cumprindo seu contrato ou falhando explicitamente. Ou seja, não é aceitável que uma rotina falhe silenciosamente. A segunda lei estabelece que o fracasso de uma rotina deve sempre gerar uma exceção para quem a chamou. Já a terceira trata das possíveis estratégias após uma falha: tentar novamente corrigindo o problema ou interromper a operação de maneira organizada.

Essa visão torna o tratamento de exceções muito mais coerente e previsível. Em vez de usar exceções como desvios arbitrários de fluxo, o autor as conecta diretamente à ideia de contrato quebrado. Isso dá sentido conceitual ao mecanismo e evita muitos dos problemas encontrados em abordagens tradicionais.

Na parte final do texto, Meyer apresenta o mecanismo utilizado pela linguagem Eiffel para lidar com exceções, baseado nas palavras-chave `rescue` e `retry`. A proposta é relativamente simples, mas bastante elegante. Quando ocorre uma falha, a rotina pode tentar restaurar um estado consistente e repetir a operação. Caso não consiga, a exceção é propagada para o chamador. Essa estratégia reforça a ideia de que o sistema deve sempre preservar sua estabilidade, mesmo diante de erros.

De maneira geral, o capítulo apresenta uma visão extremamente organizada e disciplinada da engenharia de software. O conceito de contratos transforma relações entre módulos em acordos claros, reduz ambiguidades e melhora significativamente a confiabilidade do sistema. Além disso, o texto mostra como boas práticas de especificação podem influenciar diretamente na qualidade do código, na documentação e até mesmo na manutenção futura do software.

Como reflexão final, a proposta de Design by Contract demonstra que confiabilidade em software não depende apenas de técnicas de programação, mas principalmente de clareza na definição de responsabilidades. Meyer defende uma engenharia de software mais rigorosa, na qual cada componente saiba exatamente o que deve receber e o que deve entregar. Mesmo sendo um texto técnico, o capítulo consegue apresentar suas ideias de maneira lógica e bastante convincente, mostrando como contratos podem servir como base para sistemas mais seguros, organizados e compreensíveis.